```
}
```

It prints:

```
sizeof BIT is = 4
value of bit is = -1
```

Why? Note that it is not a compiler error to attempt to find the *sizeof(BIT)* because it is a structure; had we attempted *sizeof(BIT.bit)*, that will not compile. Now, coming to the output, if we had used only one bit in the *BIT* structure, why is the *sizeof(BIT)* 4 bytes? It is because of the addressing requirement of the underlying machine. The machine might perhaps require all *structs* to start in an address divisible by 4; or perhaps, allocating the size of a *WORD* for the structure is more efficient even if the underlying machine may require that *structs* start at an even address. Also, the compiler is free to add extra bits between any struct members (including bit-field members), which is known as 'padding'.

Now let us come to the next output. We set *BIT.bit = 1;* and the *printf* statement printed -1! Why was that?

Note that we declared *bit* as *int bit : 1;* where the compiler treated the bit to be a signed integer of one bit size. Now, what is the range of a 1-bit signed integer? It is from 0 to -1 (not 0 and 1, which is a common mistake). Remember the formula for finding out the range of signed integers: $2^{(n-1)}$ *to* $2^{(n-1)}-1$ where *N* is the number of bits. For example, if *N* is 8 (number of bits in a byte), i.e., the range of a signed integer of size 8 is $-2^{(8-1)}$ to $2^{(8-1)}-1$, which is *-128 to +127*. Now, when N is 1, i.e., the range of a signed integer of size 1, it is $-2^{(1-1)}$ to $2^{(1-1)}-1$, which is -1 to 0!

No doubt, bit-fields are a powerful feature for low-level bit-manipulation. The cost of using bit-fields is the loss of portability. We already saw how padding and ending issues can affect portability in our simple program for reading the components of a floating-point number. Bit-fields should be used in places where space is very limited, and when functionality is demanding. Also, the gain in space could be lost in efficiency: bit-fields take more time to process, since the compiler takes care of (and hides) the underlying complexity in bit-manipulation to get/set the required data. Bugs associated with bit-fields can be notoriously hard to debug, since we need to understand data in terms of bits. So, use bit-fields sparingly and with care. END

By: S G Ganesh

The author works for Siemens (Corporate Technologies), Bangalore. You can reach him at *sgganesh@gmail.com*.